

C#.NET



Mr. Bangar Raju

NARESHO
technologies

Differences between Variable Instance and Reference

Understanding the Differences Between Variable, Instance, and Reference in C#

By Albert Harmon

Published September 10, 2016

🕒 4 min read

Education



TLDR This blog post explains the key differences between a variable, an instance, and a reference in C#. It covers the definitions, memory allocation, and

practical examples to illustrate how each concept works in object-oriented programming.

In this blog post, we will explore the fundamental concepts of variables, instances, and references in C#. Understanding these concepts is crucial for any programmer working with object-oriented programming. Let's break down each term and see how they relate to classes in C#.

What is a Class?

Before diving into the differences, it's essential to understand what a class is. A class is a user-defined type that serves as a blueprint for creating objects. In programming, classes can be predefined (like ``string`` or ``int``) or user-defined. Every class is essentially a data type, and you cannot use a class directly without creating an instance of it.

Variable of a Class

A variable of a class is an uninitialized copy of that class. It does not allocate memory until it is initialized. For example:

```
First f; // f is a variable of the class First, but it is unin
```



In this case, `f` is simply a variable that does not point to any memory location until it is assigned an instance of the class.

Instance of a Class

An instance of a class is a copy of the class that has been initialized using the `new` keyword. When you create an instance, memory is allocated for that instance. For example:

```
First f = new First(); // f is now an instance of the class Fi
```



Here, `f` is an instance of the class `First`, and memory has been allocated for it. You can access the members of the class through this instance.

Example of Instance Creation

To illustrate this further, consider the following code snippet:

```
class First {  
    public int x = 100;  
}  
  
class Program {  
    static void Main() {  
        First f = new First(); // f is an instance of First  
        Console.WriteLine(f.x); // Outputs: 100  
    }  
}
```

In this example, `f` is an instance of the class `First`, and we can access its member `x` because `f` has been initialized.

Reference of a Class

A reference of a class is a pointer to an existing instance of that class. Unlike an instance, a reference does not allocate its own memory; instead, it shares the memory of the instance it points to. For example:

```
First f1 = new First(); // f1 is an instance of First  
First f2 = f1; // f2 is a reference to f1
```

In this case, `f2` is a reference to the same instance as `f1`. Any changes made to the instance through `f1` will be reflected when

accessed through `f2`, and vice versa.

Example of Reference Usage

Consider the following code:

```
class First {  
    public int x;  
}  
  
class Program {  
    static void Main() {  
        First f1 = new First();  
        f1.x = 100;  
        First f2 = f1; // f2 references f1  
        f2.x = 200;  
        Console.WriteLine(f1.x); // Outputs: 200  
    }  
}
```

In this example, modifying `f2.x` also changes `f1.x` because both `f1` and `f2` point to the same instance.

Summary of Differences

To summarize:

- **Variable of a Class:** An uninitialized copy of a class that does not allocate memory until initialized.
- **Instance of a Class:** A copy of a class that is initialized using the ``new`` keyword, allocating memory for its members.
- **Reference of a Class:** A pointer to an existing instance of a class that shares the same memory as the instance it references.

Understanding these distinctions is vital for effective programming in C#. Each concept plays a crucial role in memory management and object-oriented design, ensuring that developers can create efficient and maintainable code.

Thank you for reading!